Master Thesis defense

Rémi Mercier, TSIA

Double Degree Télécom Paris & TUM

NovainSilico

Company Presentation

Biomodeling Company: Specialized in the modeling and simulation of complex biological systems.

Collaboration with Major Pharmaceutical Groups: Key partner for accelerating R&D.

Platform Development: Creation of a proprietary and innovative 'In Silico' solution.

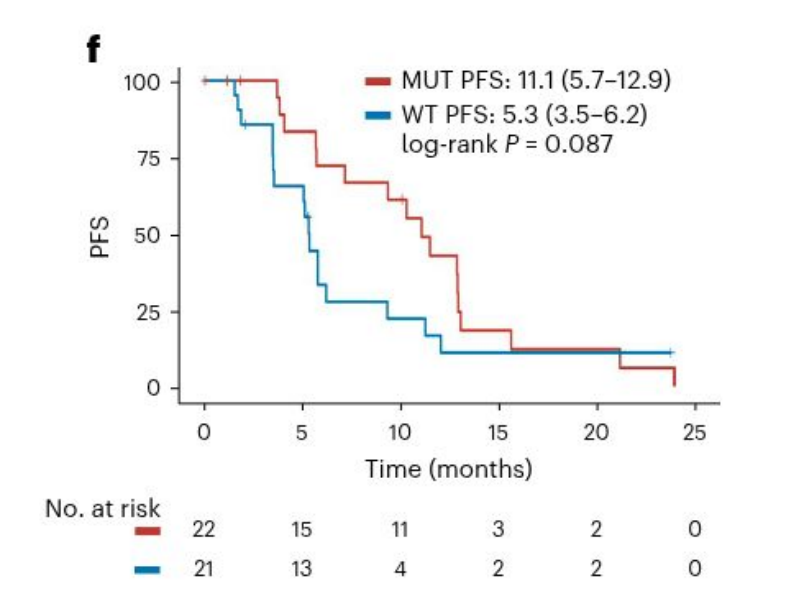
Problem:

Many plots in scientific papers

Extracting data from it takes time and effort

Output:

```
{
  "series": [
    {
      "label": "<legend name if visible else 'series_N'>",
      "points": [
        {"x": <float>, "y": <float>},
        ...
      ]
    },
    ...
  ]
}
```

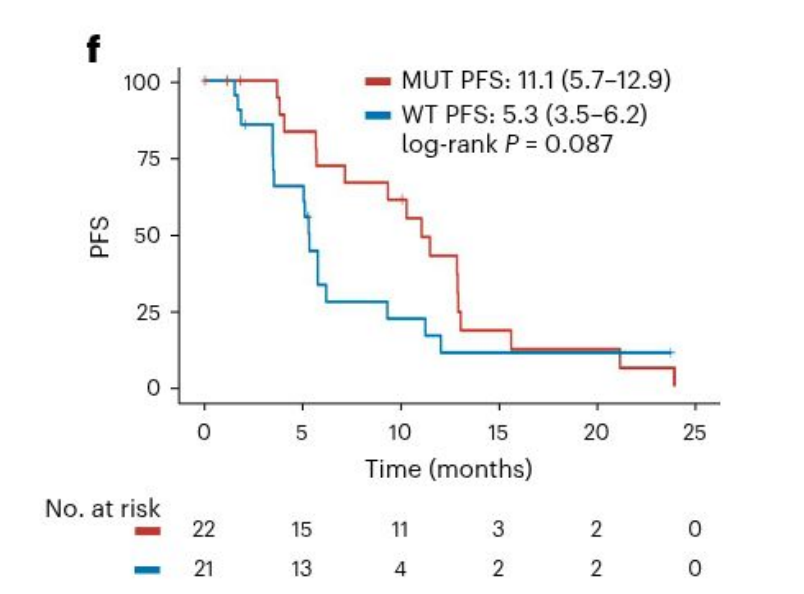


Pipeline

1. Axis Calibration
2. Router: 3 options
    a. Kaplan Meyer plots
    b. Categorical plots
    c. Other plots
3. Plotting over the base image
4. Results
5. Conclusion

1. Axis Calibration

a. Axis locations
b. OCR
c. 1st filter
d. Tick Snapping
e. 2nd filter
f. Fit the axis

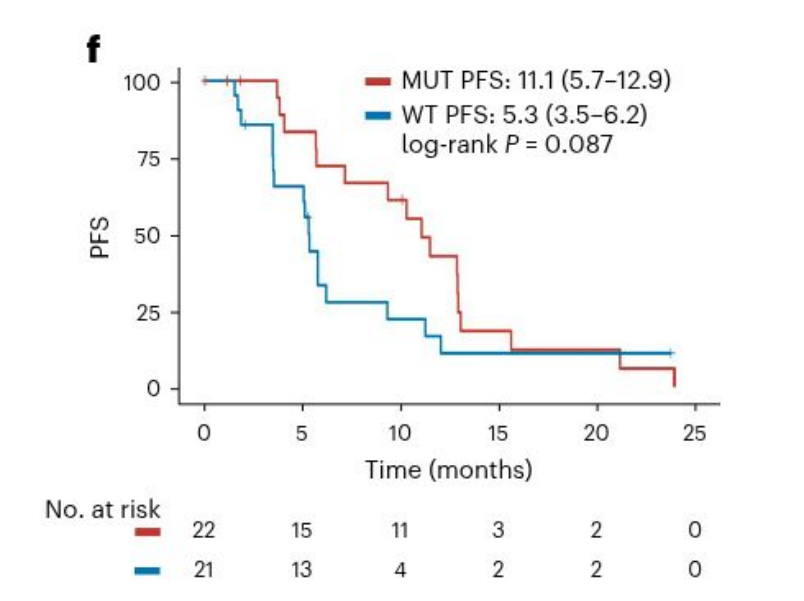


1. Axis Calibration

a. Axis locations

We use the HoughLines Transform, to get all lines

Then :
We select the best vertical and horizontal lines

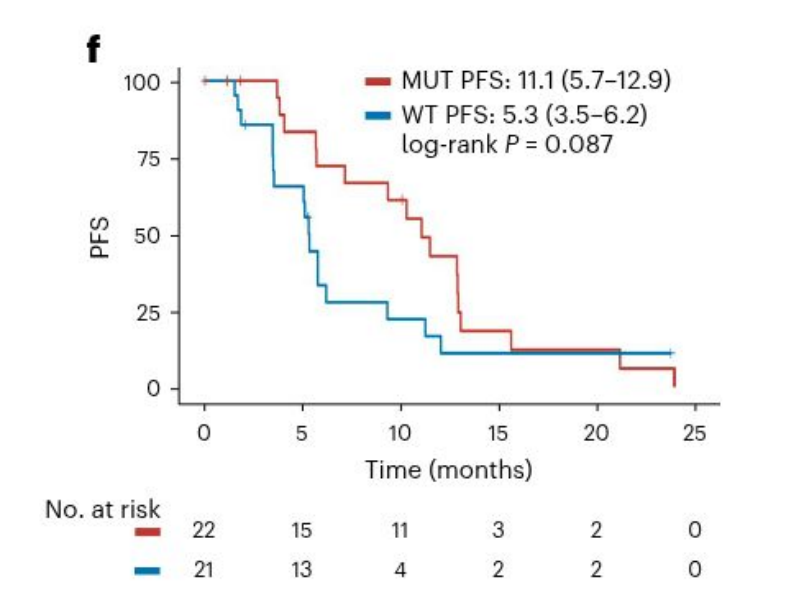


1. Axis Calibration

b. OCR

EasyOCR near the axis

We turn the axis : 0 ° ; +90° ; -90 °
To be sure not to miss any label.

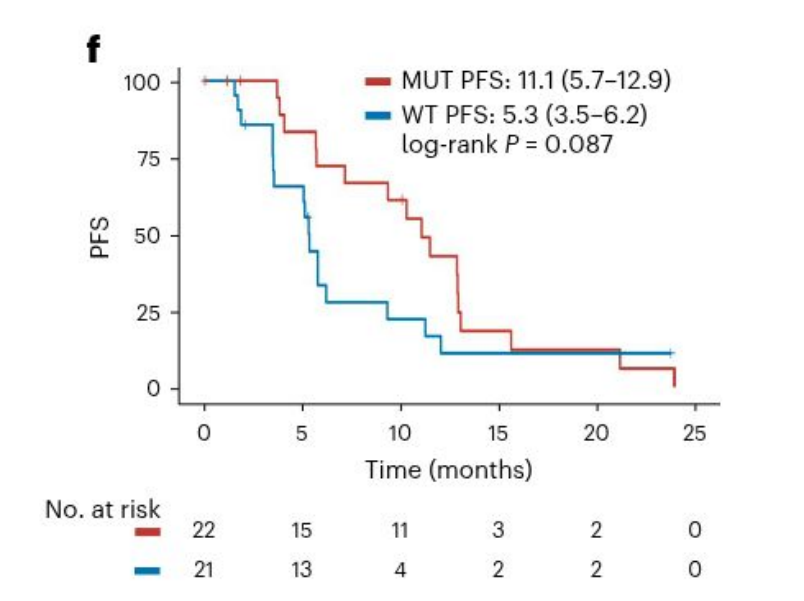


1. Axis Calibration

c. LLM Filter

LLM Call: gives us all labels of existing ticks

A true label will be in the LLM output



1. Axis Calibration

d. Tick Snapping

```python
class TickCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, 3, padding=1)

        self.fc1 = nn.Linear(64 * 4 * 4, 64)
        self.fc2 = nn.Linear(64, 2)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2)

        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2)

        x = F.relu(self.conv3(x))
        x = F.max_pool2d(x, 2)

        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        return self.fc2(x)
```

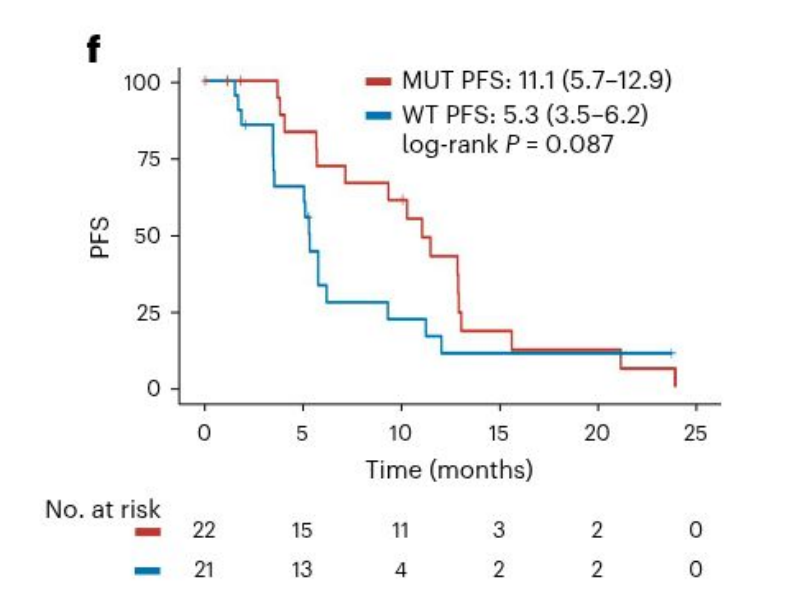
```
Precision: 0.957
Recall   : 0.900
```

Tick Snapping:

Heuristics were too brittle

TickCNN: simple DL model
Output: No tick / Tick

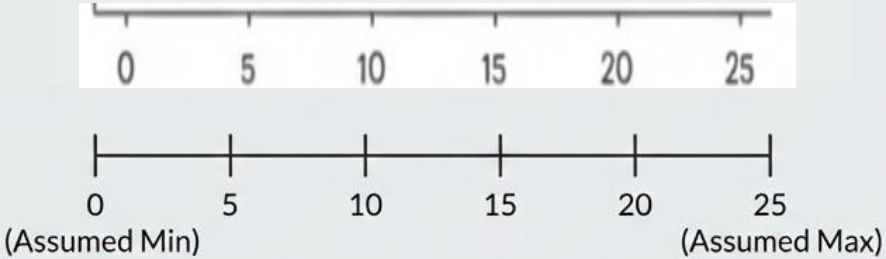
Score:
softmax(TickCNN[Tick]) + distance * bias

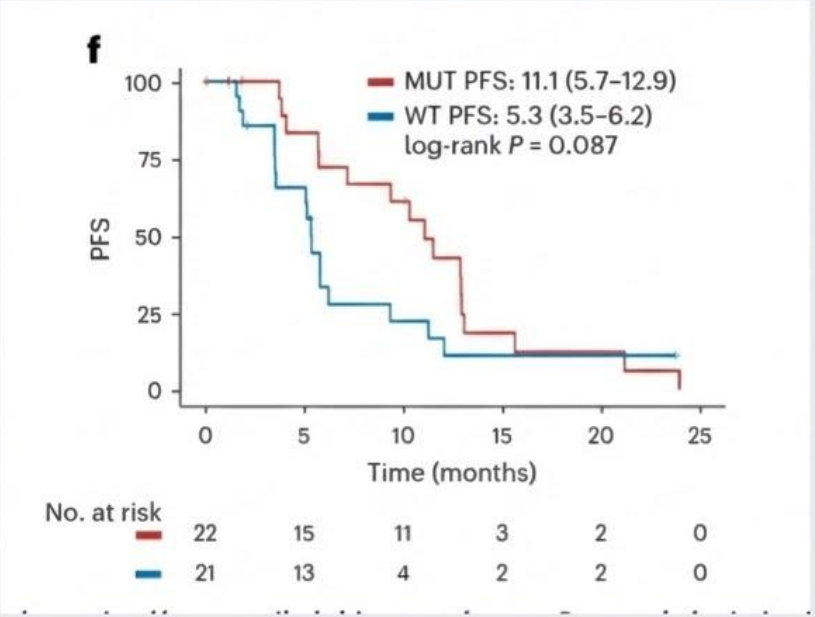


1. Axis Calibration

e. Second Filter

Remove obvious illogical labels





1. Axis Calibration

f. Calibration in itself

```python
if mode == "linear":
    return v

if mode.startswith("log"):
    if v <= 0 or base <= 0 or base == 1:
        return None
    return math.log(v, base)
```

Adjust for logarithmic axis

We select the subset maximizing this score :

$$\text{score} = \underbrace{\lambda\,|\mathcal{S}|}_{\text{Size of the subset}} - \underbrace{\alpha \cdot \mathcal{L}_{\text{Huber}}(\mathcal{S})}_{\text{Huber Loss (~MSE)}} + \underbrace{\gamma \cdot \text{span}(\mathcal{S})}_{\text{Span of the subset}},$$

Then numpy.polyfit:

The solution minimizes the squared error

$$E = \sum_{j=0}^{k} |p(x_j) - y_j|^2$$

in the equations:

```
x[0]**n * p[0] + ... + x[0] * p[n-1] + p[n] = y[0]
x[1]**n * p[0] + ... + x[1] * p[n-1] + p[n] = y[1]
...
x[k]**n * p[0] + ... + x[k] * p[n-1] + p[n] = y[k]
```

pixel = a * value + b

2. Router

Kaplan Meier like:
monotonous

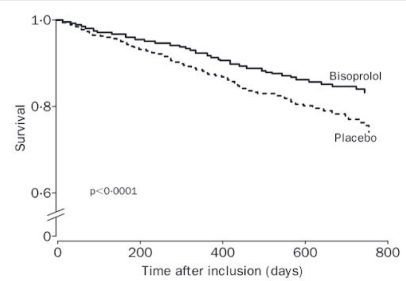
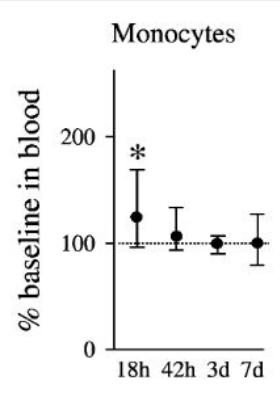
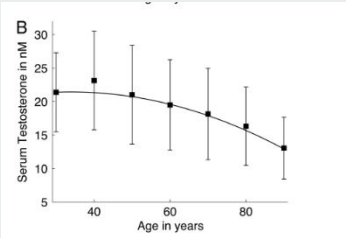


Figure 2: Survival curves

Categorical Plot:
X axis calibration is not continuous

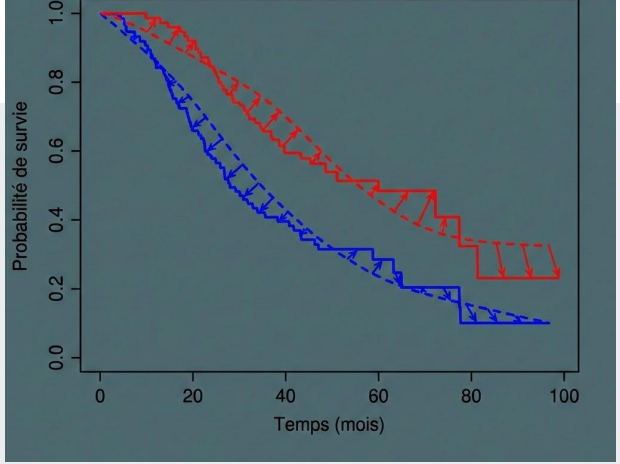


Other Plots



2. Router

a. Kaplan Meier like



A first simple extraction (LLM call)

Snapping:
-> Heuristic:
We are looking for the best beams

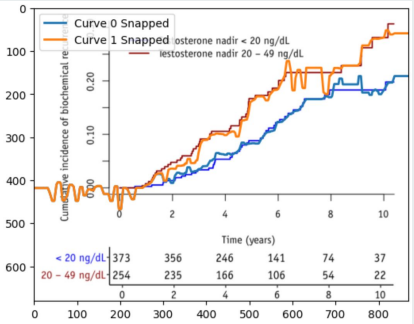
-> DL model:
promising but need more work
would help apply this for other types of graphs

Difference with LLM extraction

Smoothness loss

Discourage being close to the other series

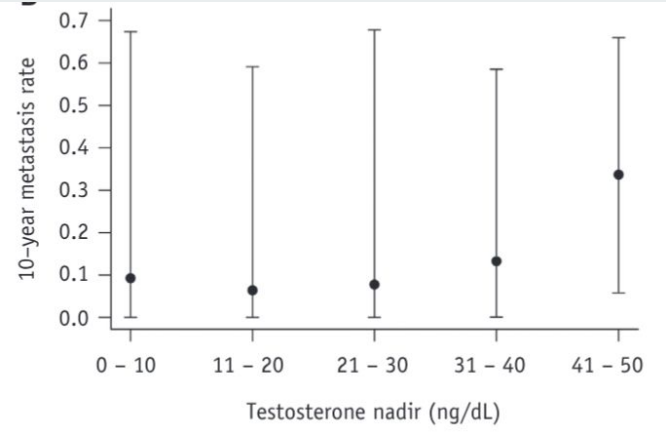
$$\text{Total Cost} = \sum_{s} \left(dy_s + \lambda |y_{x,s} - y_{x-1,s}|\right) + \sum_{i<j} w \exp(-|y_{x,i} - y_{x,j}|/\sigma)$$

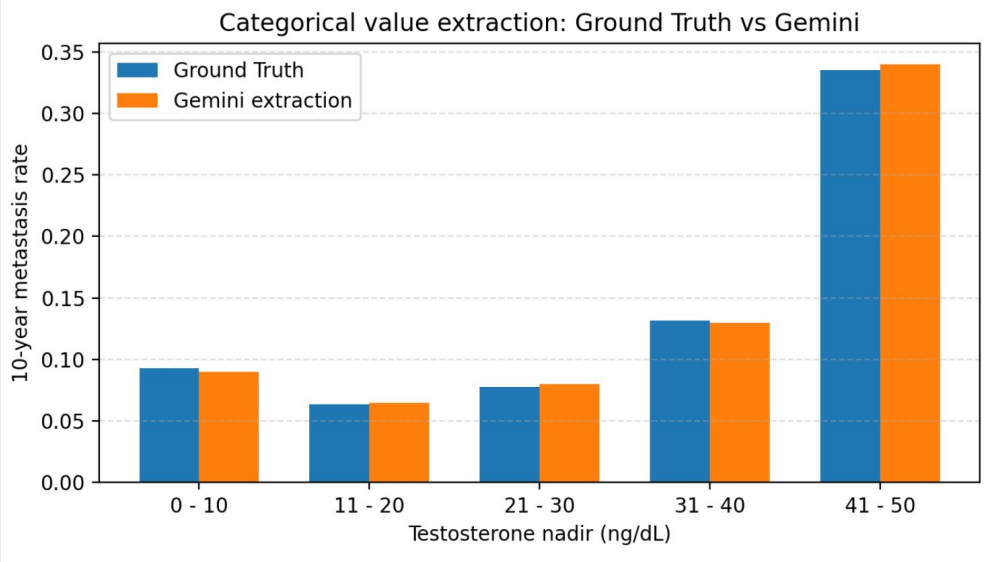


2. Router

b. Categorical Plot

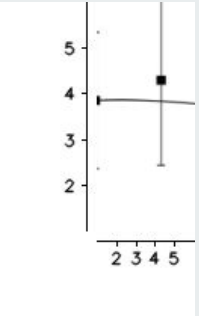
We use a single LLM call

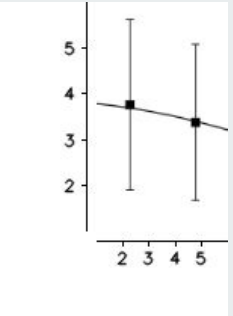




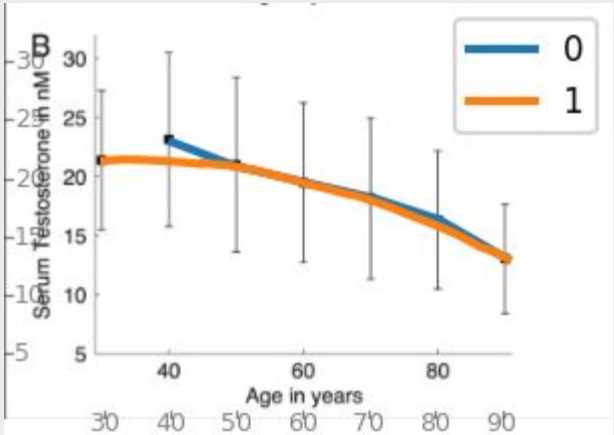
2. Router

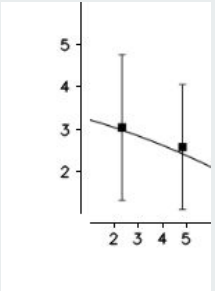
c. Other Plots

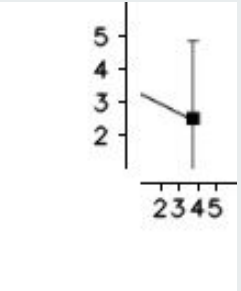




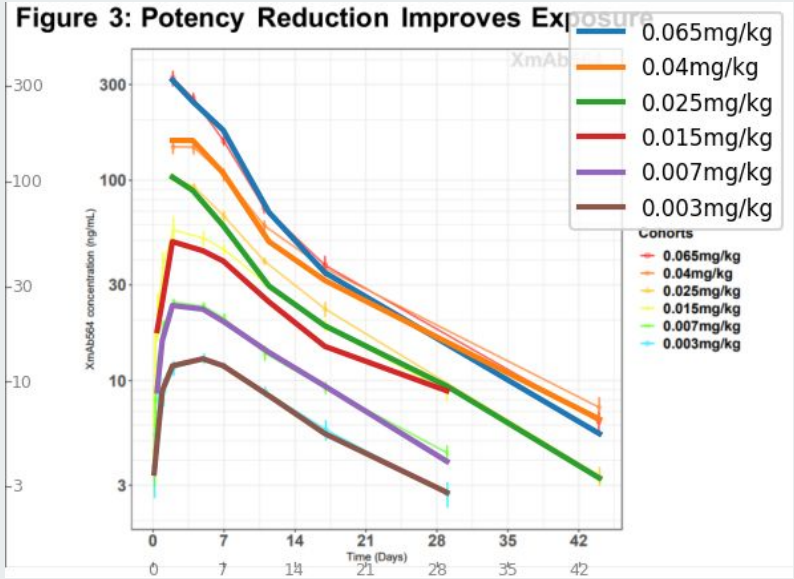
Divide and Conquer (D&C):
Simplify the task for the LLM

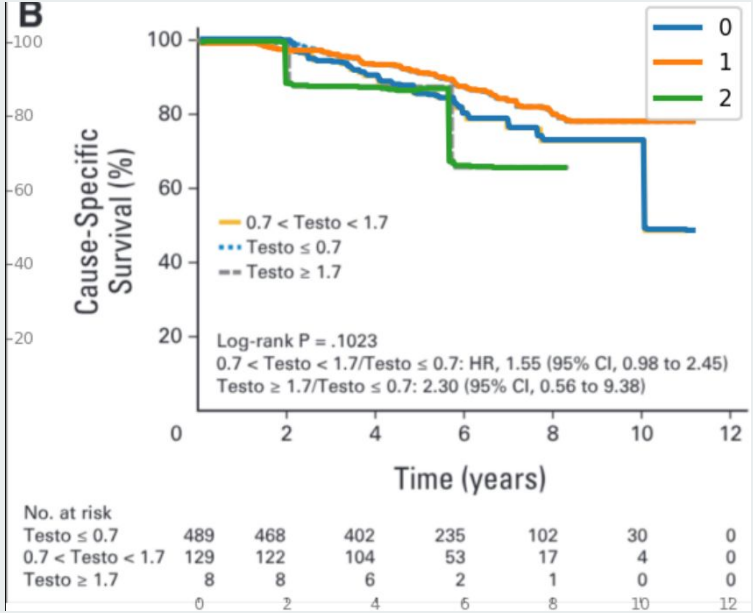






3. Plotting over the base image

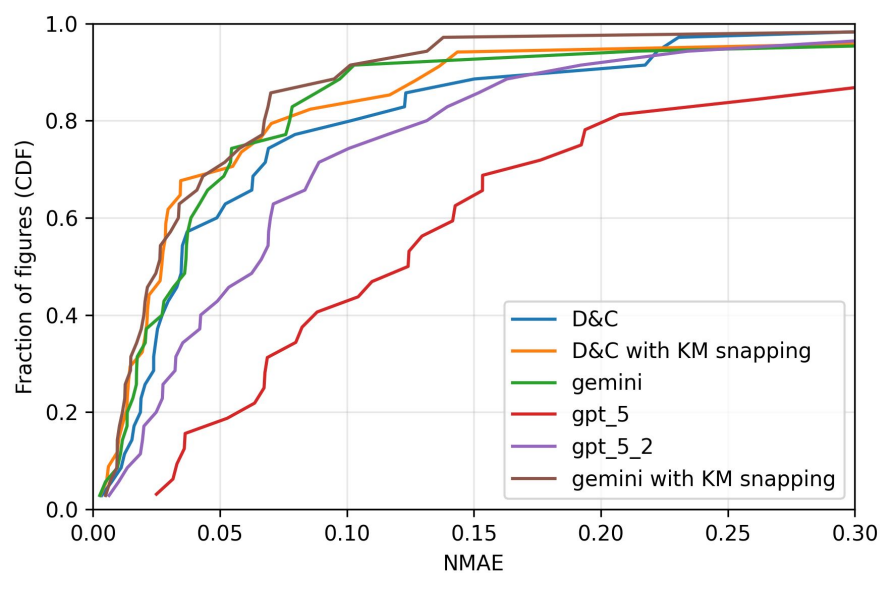


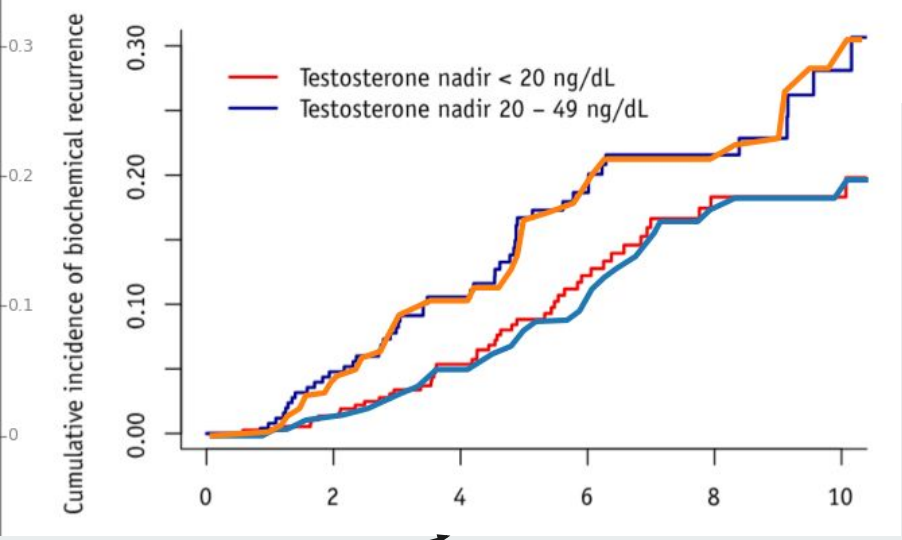


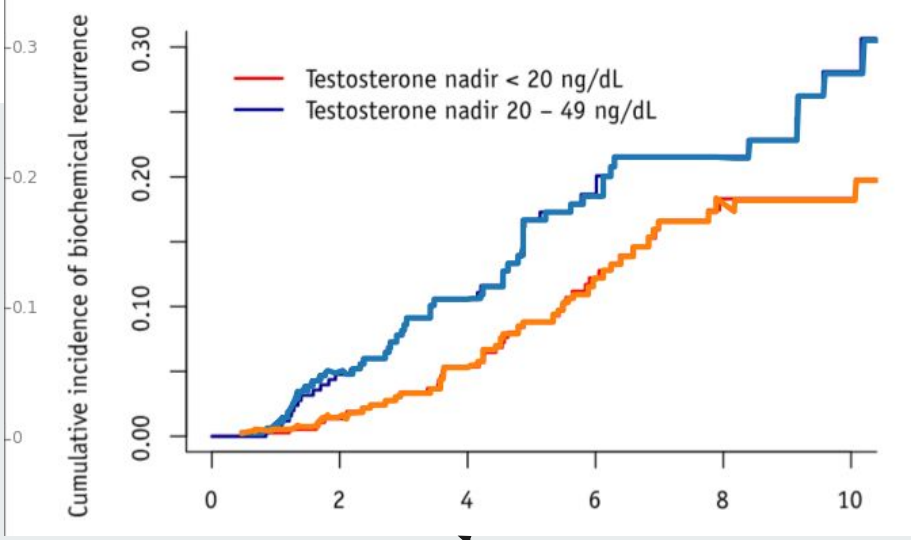
4. Results

Benchmark of 35 graphs of different types

Rank	Method	Overall Median NMAE
1	D & C + KM snapping	0.0270
2	Gemini + KM snapping	0.0284
3	D & C	0.0341
4	Gemini	0.0361
5	GPT-5.2	0.0644
6	GPT-5	0.1244



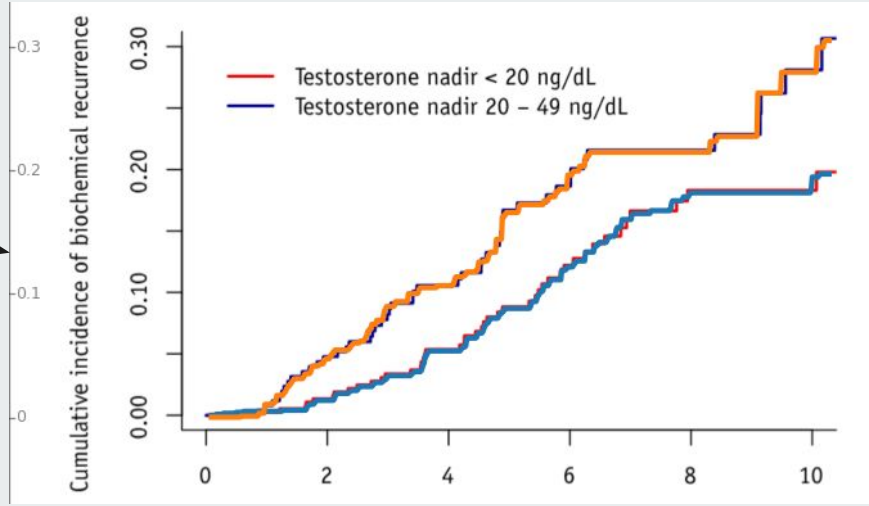




Gemini

D&C

Gemini + snapping



5. Conclusions

Main work:

- Geometric grounding of the extraction
- Snapping for KM and curves-based plots

Multimodal LLMs as router, and first pass

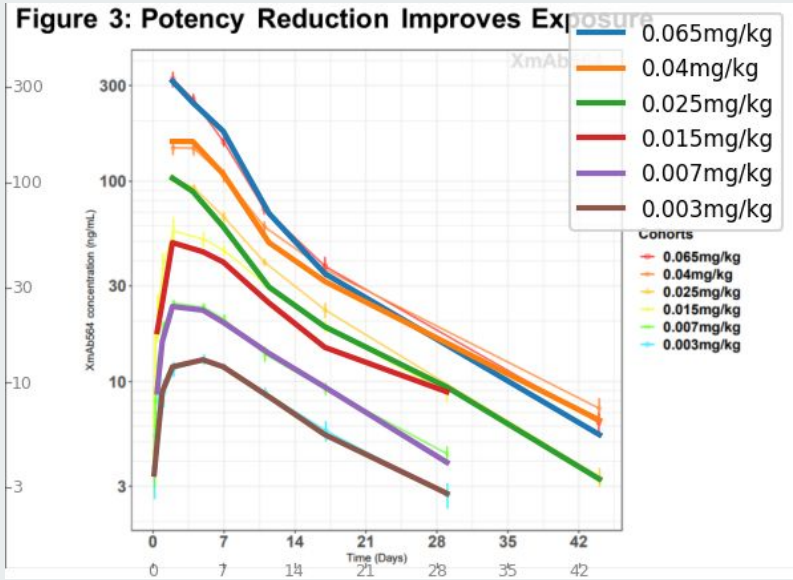
If the plot is simple, and the precision isn't that needed, a LLM call can suffice

Future improvements:

Better DL models

Post extraction processing

Post extraction processing: optimization



Use the LLM as a judge to get advice on how to improve
Another LLM call with the extracted data and the advice

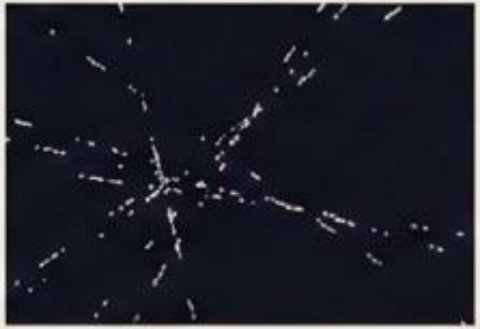
Issues:

- Correction Loop
- large context needed

Annex

Hough Line Transform: The Math

Points in the image space are transformed into polar coordinates using the equation $\rho = x\cos(\theta) + y\sin(\theta)$. We then select the best vertical and horizontal lines.



Edge Image

$\longrightarrow$

Detected Lines